Ders 20 Çalışma Özeti

Ders 20 Çalışma Özeti

Genel Konular

- Minimum yayılan ağaç (minimum spanning tree, MST) tekrarı: Prim ve Kruskal algoritmaları.
- Kruskal algoritmasının etiket dizisiyle (label array) tasarımı ve C koduna dökülmesi.
- `union` fonksiyonu ile iki alt grafın birleştirilmesi.
- Kruskal ile Prim'in karşılaştırılması: seyrek (sparse) ve yoğun (dense) graflar.
- En kısa yol (shortest path) problemi ve Dijkstra algoritması.
- Dijkstra'nın öncelik kuyruğu (priority queue) ile çalışması ve sözde kodu (pseudo code).
- Derinlik öncelikli arama (depth-first search, DFS) ve genişlik öncelikli arama (breadth-first search, BFS) bu dönem anlatılmadı. Hoca bunların önümüzdeki dönem anlatılacağını söyledi.

Hocanın Özellikle Vurguladığı Kısımlar

- Prim ve Kruskal ikisi de açgözlü (greedy) algoritmadır. Hoca mantığı "yerelde en iyi seçim, globalde de en iyi sonucu verir" diye özetledi. Bu MST için ispatlıdır (kesme özelliği, cut property); her problemde geçerli genel bir kural değildir.
- Kruskal'da ilk iş kenarları ağırlığa göre küçükten büyüğe sıralamaktır. Karmaşıklık bu sıralamadan gelir: $O(E \log V)$.
- n düğümlü bir grafın MST'sinde $n-1$ kenar vardır. Döngü bu yüzden `i < nV - 1` koşuluyla döner.
- Kruskal'da iki düğümün etiketine bakılır. Dört durum vardır:
  - İkisi de 0: ikisi de hiçbir yere bağlı değil. Kenar alınır, ikisine yeni etiket verilir.
  - Biri 0, biri x: kenar alınır, 0 olan düğüm x ile etiketlenir.
  - Biri x, biri y: kenar alınır, bütün y'ler x yapılır (iki alt graf birleşir).
  - İkisi de x: iki düğüm zaten aynı alt grafta. Kenar alınırsa döngü (cycle) oluşur. Kenar alınmaz.
- Yoğun graflarda Prim, seyrek graflarda Kruskal daha iyi çalışır. Prim bir düğüme birden çok kez bakmak zorunda kalır. Kruskal'da bir kenarla işi biten düğüme geri dönülmez.
- Dijkstra yalnızca negatif olmayan ağırlıklı graflarda çalışır. Yönlü ve yönsüz graflarda kullanılabilir.
- Dijkstra, Prim'e çok benzer. Fark: Prim tek kenar ağırlığına, Dijkstra kaynaktan toplam mesafeye bakar.
- Dijkstra'nın kullanım alanları: trafik planlama, robot navigasyonu, ağ yönlendirme (network routing) protokolleri, zamanlama (scheduling).
- Düğümlerin siyaha boyanması yalnızca görsel amaçlıdır. Kuyruktan çıkan düğümle işimiz biter; ayrı bir renk dizisi tutmak algoritmik olarak gerekmez.

Kısa Tekrar Notları

- MST: bütün düğümleri bağlayan, döngüsüz, toplam ağırlığı en küçük alt graf.
- Kruskal: kenarları sırala, en küçükten başla, döngü oluşturmayan kenarı al.
- Etiket dizisi: her düğüm bir indis. Başlangıçta hepsi 0. Aynı etiket = aynı alt graf.
- Kruskal karmaşıklığı: $O(E \log V)$ (sıralama baskın).
- Dijkstra başlangıcı: $d[s] = 0$, diğer bütün düğümler için $d[u] = \infty$, $pred[s] = NULL$.
- Gevşetme (relaxation) adımı: $d[u]+w(u,v) < d[v]$ ise $d[v] = d[u]+w(u,v)$ ve $pred[v] = u$.

- Öncelik kuyruğundan her adımda en küçük mesafeli düğüm çıkar (`extract-min`). Mesafesi değişen düğüm için `DECREASE-KEY` yapılır.
- `pred` dizisi en kısa yolu geriye doğru izlemeyi sağlar.

Detaylı Açıklamalar

Kruskal Örneği

Hoca 8 düğümlü (A-H) bir graf çizdi. Kenarlar ağırlığa göre sıralandı:

`AB4, CF5, AC6, GH7, CD8, EG9, DF10, FG11, FH14, AD16, EF18, DH21, CE23, BE24`

Etiket dizisi üzerinden adım adım ilerleme:

Kenar	Etiketler (önce)	Karar	Sonuç
AB4	A=0, B=0	Yeni etiket	A=B=1
CF5	C=0, F=0	Yeni etiket	C=F=2
AC6	A=1, C=2	Farklı etiket, birleştir	Bütün 2'ler 1 olur
GH7	G=0, H=0	Yeni etiket	G=H=3
CD8	C=1, D=0	D, C'nin etiketini alır	D=1
EG9	E=0, G=3	E, G'nin etiketini alır	E=3
DF10	D=1, F=1	Aynı alt graf, döngü oluşur	Alınmaz
FG11	F=1, G=3	Farklı etiket, birleştir	Bütün 3'ler 1 olur

FG11 alındığında bütün düğümler aynı alt grafa girer. 8 düğüm için 7 kenar alınmıştır. Toplam ağırlık:

$$4 + 5 + 6 + 7 + 8 + 9 + 11 = 50$$

Kruskal Kodu

Hoca kodu tablette yazdı. Tablettaki hâli derlenmiyor ve yanlış sonuç veriyor (bkz. ders sonrası notlar). Aşağıdaki kod hocanın yapısını koruyarak düzeltilmiş hâlidir; `gcc -Wall` ile derlendi, örnek grafta 7 kenar ve toplam 50 verdi:

```c
typedef struct Edge {
    int u, v;
    int weight;
} EDGE;

/* union C'de ayrılmış sözcük olduğu için fonksiyon adı unionLabels */
void unionLabels(int label[], int nV, int oldLabel, int newLabel)
{
    int i;
    for (i = 0; i < nV; i++)
        if (label[i] == oldLabel)       /* düğüm indisi değil, etiket karşılaştırılır */
            label[i] = newLabel;
}

/* gr: ağırlığa göre sıralı kenarlar, nE: kenar sayısı; dönüş: MST'deki kenar sayısı */
int kruskal(EDGE gr[], int nE, EDGE mst[], int nV)
{
    int label[nV];
    int labelNo = 1, i, j, uu, vv;
    for (i = 0; i < nV; i++)
        label[i] = 0;
    i = j = 0;
    while (i < nV - 1 && j < nE) {
        uu = gr[j].u;
```

```
        vv = gr[j].v;
        if (label[uu] + label[vv] == 0) {        /* ikisi de etiketsiz */
            label[uu] = label[vv] = labelNo++;
            mst[i++] = gr[j];
        }
        else if (label[uu] != label[vv]) {       /* farklı alt graflar */
            if (!label[uu])
                label[uu] = label[vv];
            else if (!label[vv])
                label[vv] = label[uu];
            else
                unionLabels(label, nV, label[vv], label[uu]);
            mst[i++] = gr[j];
        }
        j++;                                     /* etiketler eşitse kenar atlanır */
    }
    return i;
}
```

- `gr` dizisi kenarları ağırlığa göre sıralı tutar. `j < nE` koşulu, graf bağlı değilse dizinin dışına taşmayı önler.
- `labelNo` 1'den başlar. 0 değeri "etiketsiz" anlamına gelir.
- `label[uu] + label[vv] == 0` koşulu ancak ikisi de 0 ise sağlanır, çünkü etiketler negatif değildir.

Kruskal ve Prim Karşılaştırması

- Kruskal'da kenarlar sıralandığı için karmaşıklık $O(E \log V)$ olur.
- Çok bağlantılı (yoğun) graflarda Prim daha kısa sürede çalışır.
- Çok düğümlü ama az bağlantılı (seyrek) graflarda Kruskal daha iyidir.
- Hoca ağ bağlantısı uygulamalarında genelde Kruskal kullanıldığını söyledi.

Dijkstra: İlk Örnek

Hoca kaynak S ve hedef T olan 8 düğümlü bir graf çizdi. Kenar ağırlıkları:

```
S-2: 9    S-6: 14   S-7: 15
2-3: 24   6-3: 18   6-5: 30   6-7: 5
7-5: 20   7-T: 44
3-5: 2    3-T: 19
5-4: 11   5-T: 16
4-3: 6    4-T: 6
```

Adımlar:

Adım	Seçilen düğüm	Mesafe	Nereden
1	2	9	S
2	6	14	S
3	7	15	S
4	3	32	6 $(14 + 18)$
5	5	34	3 $(32 + 2)$
6	4	45	5 $(34 + 11)$
7	T	50	5 $(34 + 16)$

- 3 için adaylar: 2 üzerinden $9 + 24 = 33$, 6 üzerinden $14 + 18 = 32$. 32 seçilir.
- 5 için adaylar: 6 üzerinden 44, 7 üzerinden 35, 3 üzerinden 34. 34 seçilir.
- T için adaylar: 7 üzerinden 59, 3 üzerinden 51, 4 üzerinden 51, 5 üzerinden 50. 50 seçilir.

En kısa yol: S → 6 → 3 → 5 → T, toplam 50.

Dijkstra: İkinci Örnek (Tablo ile)

Yönlü graf ve komşuluk listesi (adjacency list):

```
S: A(2), B(7)
A: B(3), C(8), D(5)
B: A(2), C(1)
C: D(4)
D: C(5)
```

Her düğüm için üç bilgi tutulur: mesafe (`d`), önceki düğüm (`pred`) ve renk (beyaz: seçilmedi, siyah: seçildi).

Adım	Kuyruktan çıkan	d[A]	d[B]	d[C]	d[D]
Başlangıç	-	∞	∞	∞	∞
1	S (0)	2	7	∞	∞
2	A (2)	2	5	10	7
3	B (5)	2	5	6	7
4	C (6)	2	5	6	7
5	D (7)	2	5	6	7

- C çıktığında C'den D'ye yol $6 + 4 = 10$ olur. 10, 7'den büyük olduğu için D güncellenmez.

Son durum:

Düğüm	S	A	B	C	D
d	0	2	5	6	7
pred	NULL	S	A	B	A

- Örnek: C'ye en kısa yol S → A → B → C, toplam $2 + 3 + 1 = 6$.

Gevşetme Adımının Mantığı

- A'dan B'ye doğrudan mesafe 30, A'dan C'ye 15 ve C'den B'ye 8 olsun.
- C seçildikten sonra kontrol: $15 + 8 = 23 < 30$.
- B'nin mesafesi 23'e düşer. B'nin `pred` değeri artık C olur. Öncelik kuyruğunda B'nin anahtarı da güncellenir.

Dijkstra Sözde Kodu

```
Dijkstra(G, s, w)
{
    for (each u ∈ V) {
        d[u] = ∞;
        color[u] = white;
    }
    d[s] = 0;
    pred[s] = NULL;
    PQ = (queue with all vertices)
    while (non-Empty(PQ)) {
        u = extract-Min(PQ)
        for (each v ∈ Adj[u]) {
            if (d[u] + w(u,v) < d[v]) {
                d[v] = d[u] + w(u,v);
                DECREASE-KEY(PQ, v, d[v]);
                pred[v] = u;
            }
        }
        color(u) = BLACK;
```

```
    }
}
```

- Öncelik kuyruğu tam sıralı tutulmaz. Her adımda yalnız en küçük mesafeli düğüm çıkar. Mesafeler sonraki adımlarda değişebileceği için bu yeterlidir.

Ders sonrası notlar

- **Hocanın kodu derlenmez.** Tablettaki kodda birleştirme fonksiyonunun adı `union`'dır; `union` C'de ayrılmış sözcük olduğu için fonksiyon adı olamaz (`gcc` "expected '{' before '(' token" hatası verir). Yukarıda `unionLabels` adı kullanıldı. Ayrıca `label` dizisi ile `uu`, `vv` değişkenleri tanımlanmamıştı.
- **`union` etiket yerine indis karşılaştırıyor.** Hocanın çağrısı `union(label, nV, uu, vv)`, fonksiyon içindeki karşılaştırma `label[i] == uu` idi. `uu` ve `vv` düğüm indisleridir, etiket değeri değildir. Doğrusu etiket değerlerini göndermektir (yukarıdaki `unionLabels(label, nV, label[vv], label[uu])`). Değerler parametre olarak kopyalandığı için döngü sırasında `label[vv]`'nin değişmesi karşılaştırmayı bozmaz. Hocanın tahtadaki örneği (uu=1, vv=3) aslında etiket değerleriyle anlatılmıştır.
- **Harfiyen çalıştırılınca sonuç yanlış.** Hocanın kodu yalnız `union` adı değiştirilip eksik tanımlar eklenerek bu ders örneğinde çalıştırıldı: AB4, CF5, AC6, AD16, BE24 alındı, yani 5 kenar ve toplam 55 çıktı; döngü 7 kenara ulaşamadığı için kenar dizisinin sonunu da aştı. Düzeltilmiş kod 7 kenar ve toplam 50 verir.
- **`if` zinciri.** Hocanın kodunda ikinci `if (!label[vv])` ayrı bir `if` idi; `label[uu]` 0 iken ona etiket verildikten sonra `else` kolu da çalışıp gereksiz bir birleştirme çağırıyordu. Yukarıda `else if` zinciri kullanıldı.
- **Kenarlar sıralı olmalı.** Kenarlar fonksiyona girmeden önce ağırlığa göre sıralanmış olmalıdır.
- **Kruskal karmaşıklığı.** Sıralama $O(E \log E) = O(E \log V)$ sürer. Bu koddaki `union` her çağrıda bütün etiket dizisini taradığı için $O(V)$ maliyetlidir; en fazla $V - 1$ birleştirme olduğundan bu kısım $O(V^2)$ tutar. Disjoint-set (union-find) yapısı kullanılırsa birleştirme neredeyse sabit sürede yapılır.
- **Dijkstra ve Turing Ödülü.** Hoca algoritmanın 1972'de ortaya konduğunu ve Dijkstra'nın Turing Ödülü'nü bu algoritmayla kazandığını söyledi. Algoritma 1956'da tasarlanmış, 1959'da yayımlanmıştır. Dijkstra 1972 Turing Ödülü'nü programlama dillerine ve yapısal programlamaya genel katkıları için almıştır.
- **Dijkstra koşulu.** Hoca kenarların "pozitif" olması gerektiğini söyledi. Doğrusu "negatif olmayan" ağırlıktır; 0 ağırlıklı kenarlar sorun çıkarmaz. Negatif ağırlıklar için Bellman-Ford kullanılır.
- **İlk örnekte 3'e uzaklık.** Hoca önce 3'e 6 üzerinden gidilemeyeceğini söyledi. Bir öğrenci 6-3 kenarını hatırlattı ve hoca düzeltti: $14 + 18 = 32$, 2 üzerinden yol ise 33'tür.
- **İkinci örnekte `pred` değeri.** Hoca bir yerde "A'ya D'den gelindi" dedi. Doğrusu "D'ye A'dan gelindi" ($2 + 5 = 7$); hoca hemen ardından bunu doğru yazdı.
- **Kenar listesinde CE ve BE.** Hoca sıralı listeyi yazarken son kenarı önce "DE24" dedi. Öğrencinin uyarısıyla doğrusunun BE24 olduğu, CE'nin de 23 olduğu düzeltildi.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.